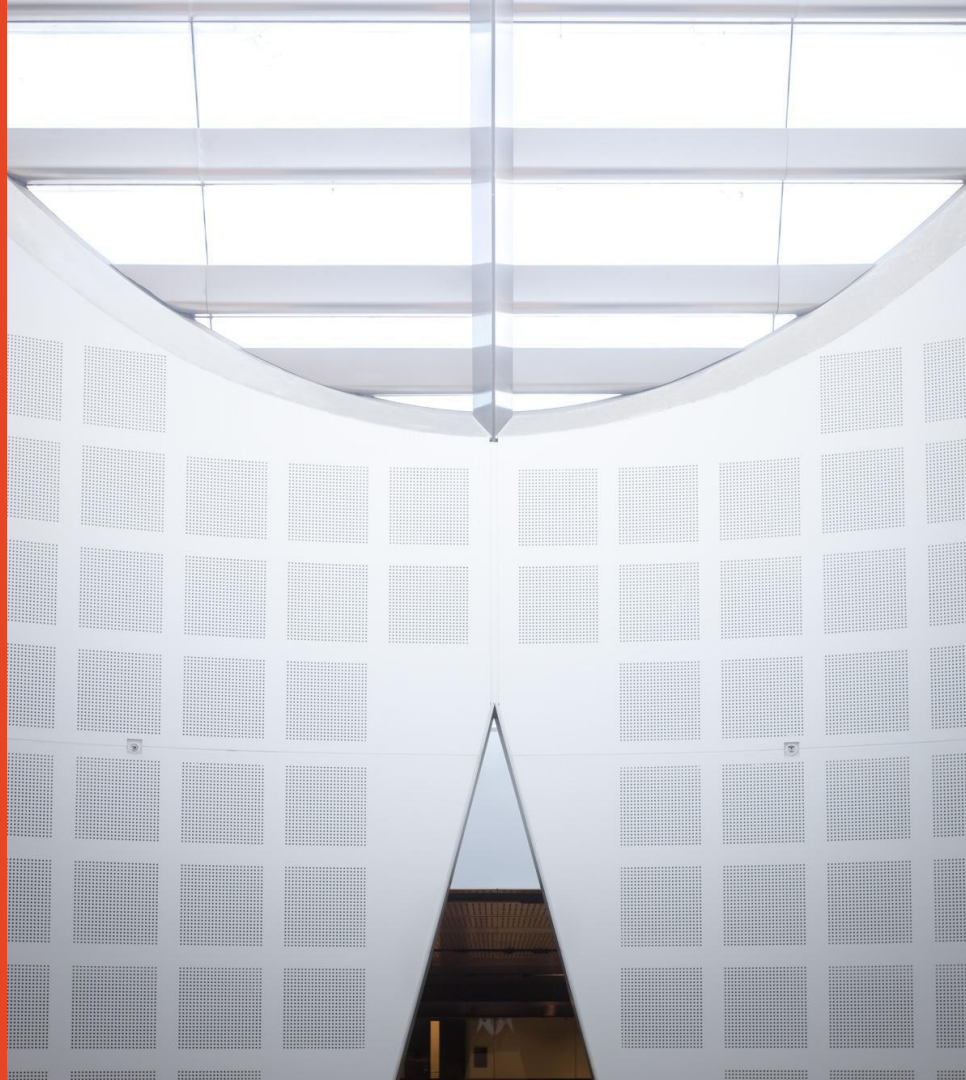
Human-AI annotation with provenance

Presented by Group CS-57

Hanchen Wang, Jingwei Lin, Yi Geng, Kanishka Kathait, Michael Max, Parth, Dishank Aswal, Tim Chung

School of Computer Science · Client: Hunter Xu from Arc Intelligence (hej)

THE UNIVERSITY OF SYDNEY



Information

Hanchen Wang (UniKey: hwan9127)

- Project manager and client liaison; review integrity (D4–D6), work queues (D8), evaluation harness (I1, I2), performance (A6)

Jingwei Lin (UniKey: [TBD])

- Review policy and cross-validation (B2, D1–D3, D9); permissions (G1–G3); agreement and evaluation (I2–I5); organisations (J1, J4)

Yi Geng (UniKey: [TBD])

- AI provider (C1); disputes and adjudication (E1–E5); Expert Gate and roles (G1, G2); history graph and release screens (F5, H6)

Kanishka Kathait (UniKey: [TBD])

- Work queues (D8); dataset intake (B5); release artefact (H1); findings record (I5)

Michael Max (UniKey: [TBD])

- AI first-pass worker (C1–C3); provenance events (F2); export policy and release gate (H2, H3)

Parth (UniKey: [TBD])

- PostgreSQL and CI; blind second review (D7); provenance capture (F1); disputes (E1–E3, E6); export checks (H5)

Dishank Aswal (UniKey: [TBD])

- Task lifecycle and completion (B4–B6); status validation (G4); reviewer corrections (D3); canonical export (H4, H5)

Tim Chung (UniKey: [TBD])

- Project setup (B1); invitations (G5); organisation roles and dashboard (J2, J3)

A dataset must explain how each label got there

- Annotation tools are built around the moment of labelling
- Dataset production is everything else: source, guideline version, reviews, AI suggestions, disagreement, release
- Without that record a dataset can be shipped, but not defended

"How did this label get here?"

A released dataset should answer this for every item, and refuse to release when it cannot

Objective: one governed journey, six pillars

1 Govern
Organisations, projects, roles, capability-based permissions, versioned guidelines

2 Assist
AI suggestions behind a typed, replaceable model interface

3 Record
Source, guideline version and the human response to AI, captured as it happens

4 Adjudicate
Review, disagreement and expert adjudication kept as distinct facts

5 Release
Immutable releases with manifests, gated before they exist

6 Evaluate
Human-only versus AI-assisted work under repeatable conditions

Related work: where existing approaches stop

Area	Representative work [cite]	What is missing for us
Annotation platforms	Label Studio, Prodigy, Argilla, CVAT [cite]	Strong labelling UI; review history and release gating are thin or add-ons
Provenance and dataset documentation	W3C PROV-DM; Datasheets for Datasets; Data Cards [cite]	Describe what to record; do not enforce it inside the workflow
Annotation agreement	Fleiss' kappa; Krippendorff's alpha [cite]	Measure agreement; say nothing about how a dispute was settled
Human-AI collaboration	Automation bias and anchoring studies [cite]	Motivate blind-then-reveal; rarely built into production tools

Background: the platform we inherited

30
defects found in the inherited code

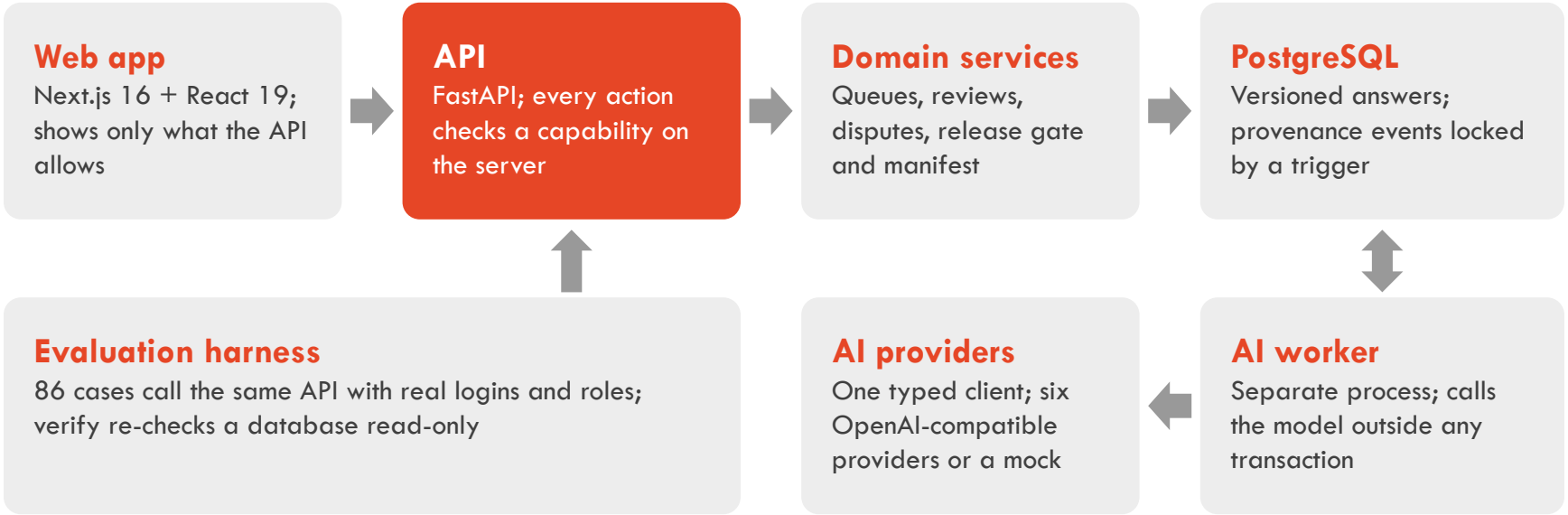
6
of them Critical, e.g. a finalised answer silently overwritten

63
user stories across 11 epics, agreed with the client

7
weekly sprints, 7 Sep – 28 Oct, including the break

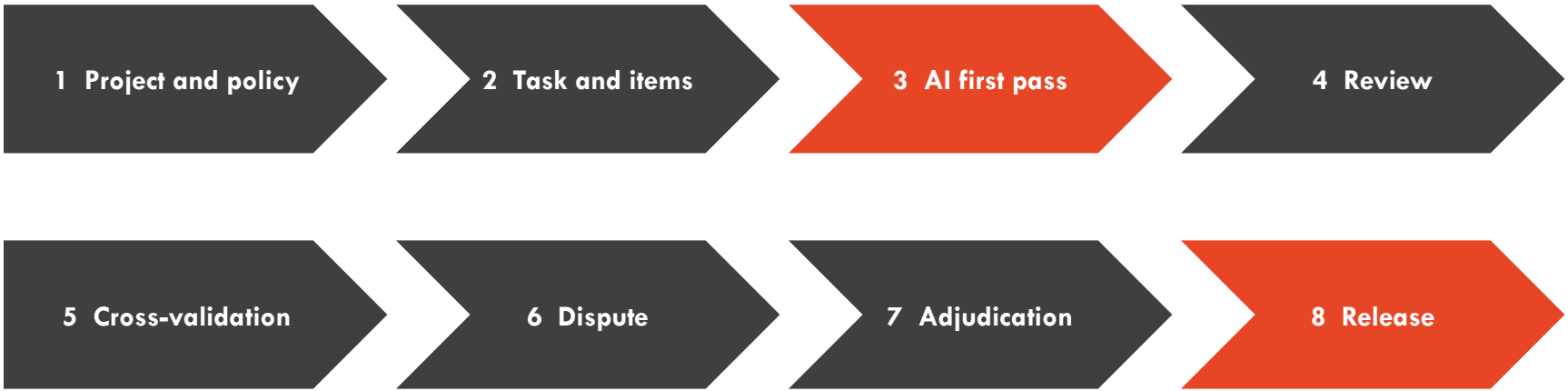
Stack we extend, not replace: Next.js 16 + React 19 web app, FastAPI + SQLAlchemy service, PostgreSQL (SQLite still supported), OpenAI-compatible multi-provider AI client

Approach: the API decides, the database remembers



No rule lives in the browser: the API checks every action, and AI suggestions and human answers land in the same versioned history.

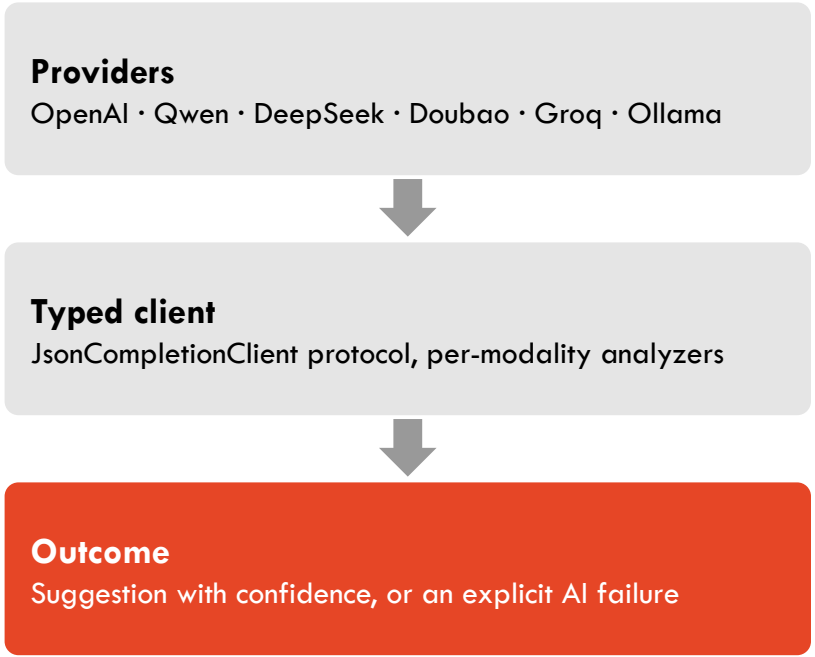
Approach: the end-to-end journey



Each step is recorded with an actor and a time, so any released item can show its source, guideline version, AI suggestion, human response, reviews and adjudication.

Approach: AI assists, and never fakes a suggestion

- One typed interface for every provider; mock and live are interchangeable
- A failed call is recorded as a failure, never as an answer
- Human-only and AI-first modes; blind-then-reveal as an evaluation protocol



Approach: independent judgement, chosen per project

- No one reviews or adjudicates their own work
- A second review never returns to the first reviewer
- A reviewer's correction is a proposal, not the answer
- An expert's decision is final: Accept, Return or Reject

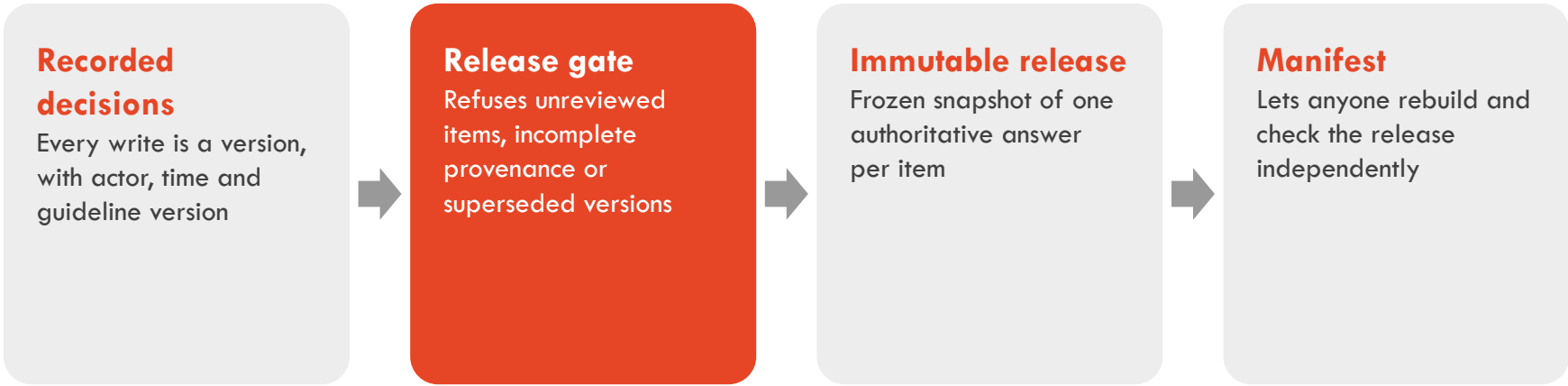
Standard
One approval; a cross-review sample (10% by default)

Dual sign-off
Two independent approvals on every item

Arbitration-ready
A dispute closes only by an independent expert

Expert Gate
An expert confirms every item before it is final

Approach: provenance and a gated, checkable release



The gate reads what was recorded; it can never be satisfied by someone asserting that review happened.

Results: the evaluation casebook

86
cases in the casebook, run through the real API

85
pass on main; one skipped, none failing

8/9
adversarial categories with a running case

30/30
inherited defects closed, the 6 Critical included

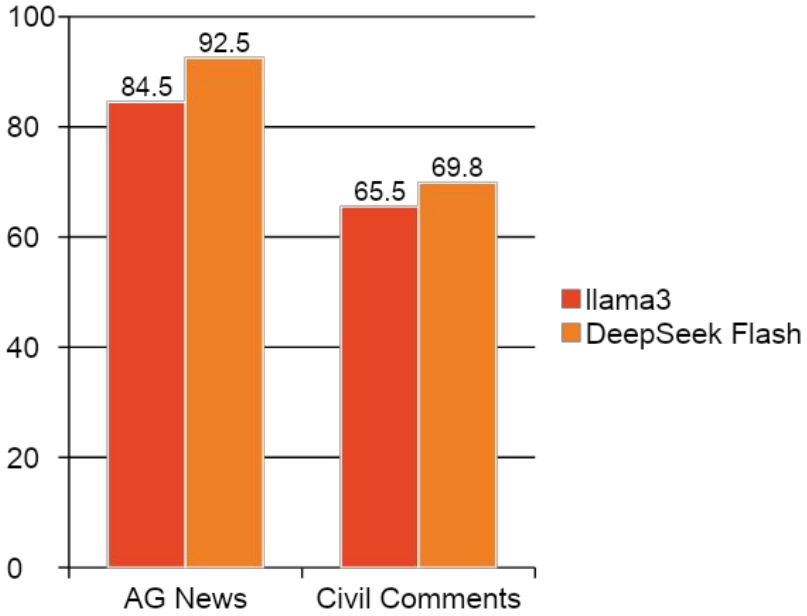
Repeatable by design: each case builds a fresh database and a scripted AI, so a run is compared with a committed baseline. Figures from main at fd35571, 22 Oct 2026.

Results: the full journey runs on realistic data

[TBD] Screenshot: one released item's history graph

- Datasets: AG News and Civil Comments ([TBD] items)
- Journey completed end to end on [TBD]
- [TBD] releases built, each with a manifest
- [TBD: one more result line]

Results: AI quality against gold labels



- AG News: DeepSeek Flash 92.5% vs llama3 84.5%
- Civil Comments: 69.8% vs 65.5%, hard for both
- Toxic comments missed: recall 47% and 35%, yet confident (ECE 0.22, 0.27)
- One malformed reply counted as an AI failure, not an answer

Accuracy (%), full recorded runs of 200 records per dataset (ADR 019); DeepSeek Flash on Civil Comments: 199 answered

Discussion: what the evidence shows

Supported

- Governance rules hold under adversarial cases
- Every release can be rebuilt from its manifest
- [TBD]

Negative results, reported

- Models are confidently wrong on the hard class ([TBD])
- Coarse confidence values flatter calibration
- Human figures come from scripted participants

Discussion: design decisions and their costs

Decision	Why	Cost
No item assignment	Client decision: work is self-served from role-checked queues	Independence must live in the queue query
PostgreSQL by default, SQLite kept	Concurrency for batch and multi-user work; fast local dev	Two databases to test; CI runs both
A correction is a proposal	The annotator's answer and the reviewer's view stay distinct	An expert or the author must adopt it
Admin oversees, does not judge	Separates governance from judgement	More roles to set up per project
Text is the committed demo path	Depth over breadth in the time available	Image, audio and video only must not break

Conclusion: against the brief's success criteria

Expected outcome	Status	Evidence
Full journey runs on realistic data	[TBD]	[TBD]
End-to-end provenance for every released item	[TBD]	[TBD]
Immutable releases with manifests, gated before they exist	[TBD]	[TBD]
AI behind a typed interface; failure never disguised	[TBD]	[TBD]
Evaluation harness and a 40–60 case casebook	[TBD]	[TBD]
Every Critical defect closed or argued	[TBD]	[TBD]

Reflection: what worked, what we would change

What worked

- One spec per story; 22 decision records
- Independent review on every pull request
- The casebook as a regression net

What we would change

- Records drifted ahead of the code
- Parallel branches changed the same files
- [TBD: one lesson per member]

Appendix

References

[TBD] Full citations for the Related work slide, verified

Architecture decision records

001 Explicit governance posture fields
002 Policy layer precedence
003 Review justification always required
004 Task-level annotation surfaces
005 Record-based text intake
006 Evaluation harness drives the API
007 Review decisions follow the queue
008 Reopening supersedes the round
009 Every write is a version
010 Independent second review
011 Adjudication independence
012 A reviewer correction is a proposal
013 Annotation-level disputes
014 Review and submission state history
015 Production history at decision time
016 Blind-then-reveal is an evaluation protocol
017 Expert Gate reuses item reconciliation
018 Guidelines and policies are versioned
019 Agreement coefficients and a recorded model run
020 The release gate reads recorded decisions
021 A manifest lets anyone check the release
022 A held disagreement may be settled

Q&A backup

[TBD] Backup material: defect register status, permission model, casebook coverage